

Unit -1 Introduction to C#.Net

- 1.1 Introduction to C#.net
- 1.2 Development of C#.net
- 1.3 Introduction to C#.net and it's features
- 1.4 Structure
- 1.5 variable
- 1.6 Identifiers
- 1.7 Keywords
- 1.8 Data types
- 1.9 Type conversion
- 1.10 Operators

What is C#

C# is pronounced as "C-Sharp". It is an object-oriented programming language provided by Microsoft that runs on .Net Framework.

By the help of C# programming language, we can develop different types of secured and robust applications:

- Window applications
- Web applications
- Distributed applications
- Web service applications
- Database applications etc.

C# is approved as a standard by ECMA and ISO. C# is designed for CLI (Common Language Infrastructure). CLI is a specification that describes executable code and runtime environment.

C# programming language is influenced by C++, Java, Eiffel, Modula-3, Pascal etc. languages.

C# History of Development of C#

History of C# language is interesting to know. Here we are going to discuss brief history of C# language.

C# is pronounced as "**C-Sharp**". It is an object-oriented programming language provided by **Microsoft** that runs on **.Net Framework**.

Anders Hejlsberg is known as the **founder of C# language**.



It is based on **C++ and Java**, but it has many additional extensions used to perform component oriented programming approach.

C# has evolved much since their first release in the year **2002**. It was introduced with **.NET Framework 1.0** and the current version of C# is 5.0.

Let's see the important features introduced in each version of C# are given below.

C# Version History

Version	Features	Year of release
C# 1.0	Basic Features	2002
C# 2.0	Generics, Partial types, Anonymous methods, Nullable types, Static classes	2005
C# 3.0	Var, LINQ, Lambda expression, Auto-implemented properties, Anonymous types, Extension methods	2007
C# 4.0	Dynamic binding, Named and Optional arguments	2010
C# 5.0	Asynchronous methods, Caller info attributes	2012
C# 6.0	Auto-property initializers, Null-propagating operator, Exception filters, Using static members, ...	2015

C# .NET is a popular and versatile programming language that's an integral part of the Microsoft .NET framework. C# was developed by Microsoft and released in 2000 as a part of their initiative to provide a unified platform for building a wide variety of software applications. It's known for its simplicity, strong type system, and robust support for object-oriented programming. C# .NET is widely used for developing desktop applications, web applications, games, and more.

.NET Framework



.NET is a framework to develop software applications. It is designed and developed by Microsoft and the first beta version released in 2000.

It is used to develop applications for web, Windows, phone. Moreover, it provides a broad range of functionalities and support.

This framework contains a large number of class libraries known as Framework Class Library (FCL). The software programs written in .NET are executed in the execution environment, which is called CLR (Common Language Runtime). These are the core and essential parts of the .NET framework.

This framework provides various services like memory management, networking, security, memory management, and type-safety.

The .Net Framework supports more than 60 programming languages such as C#, F#, VB.NET, J#, VC++, JScript.NET, APL, COBOL, Perl, Oberon, ML, Pascal, Eiffel, Smalltalk, Python, Cobra, ADA, etc.

Following is the .NET framework Stack that shows the modules and components of the Framework.

The .NET Framework is composed of four main components:

1. Common Language Runtime (CLR)
2. Framework Class Library (FCL),
3. Core Languages (WinForms, ASP.NET, and ADO.NET), and
4. Other Modules (WCF, WPF, WF, Card Space, LINQ, Entity Framework, Parallel LINQ, Task Parallel Library, etc.)

.NET APIs for Store/UWP apps		Task-Based Async Model	4.5 2012	
Parallel LINQ		Task Parallel Library	4.0 2010	
LINQ		Entity Framework	3.5 2007	
WPF	WCF	WF	Card Space	3.0 2006
WinForms	ASP.NET	ADO.NET	.NET Framework 2.0 2005	
Framework Class Library				
Common Language Runtime				

CLR (Common Language Runtime)

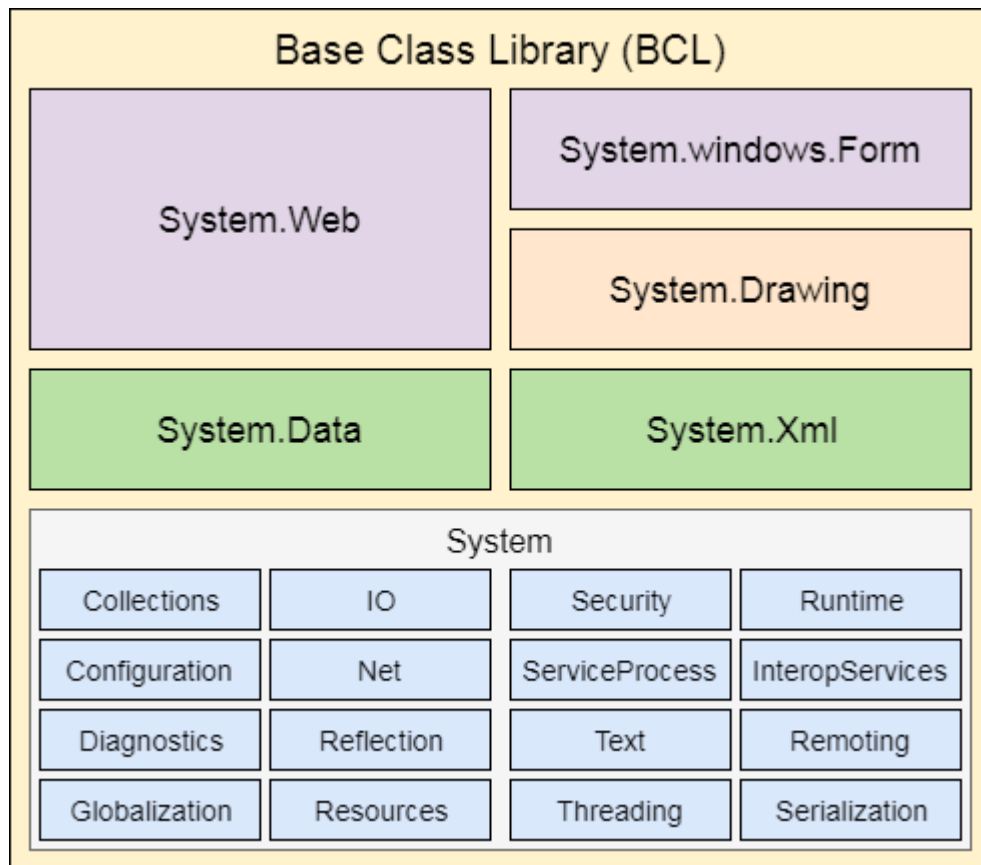
It is a program execution engine that loads and executes the program. It converts the program into native code. It acts as an interface between the framework and operating system. It does exception handling, memory management, and garbage collection. Moreover, it provides security, type-safety, interoperability, and portability. A list of CLR components are given below:

Converting Source Code into Native Code



FCL (Framework Class Library)

It is a standard library that is a collection of thousands of classes and used to build an application. The BCL (Base Class Library) is the core of the FCL and provides basic functionalities.



WinForms

Windows Forms is a smart client technology for the .NET Framework, a set of managed libraries that simplify common application tasks such as reading and writing to the file system.

ASP.NET

ASP.NET is a web framework designed and developed by Microsoft. It is used to develop websites, web applications, and web services. It provides a fantastic integration of HTML, CSS, and JavaScript. It was first released in January 2002.

ADO.NET

ADO.NET is a module of .Net Framework, which is used to establish a connection between application and data sources. Data sources can be such as SQL Server and XML. ADO .NET consists of classes that can be used to connect, retrieve, insert, and delete data.

WPF (Windows Presentation Foundation)

Windows Presentation Foundation (WPF) is a graphical subsystem by Microsoft for rendering user interfaces in Windows-based applications. WPF, previously known as "Avalon", was initially released as part of .NET Framework 3.0 in 2006. WPF uses DirectX.

WCF (Windows Communication Foundation)

It is a framework for building service-oriented applications. Using WCF, you can send data as asynchronous messages from one service endpoint to another.

WF (Workflow Foundation)

Windows Workflow Foundation (WF) is a Microsoft technology that provides an API, an in-process workflow engine, and a rehostable designer to implement long-running processes as workflows within .NET applications.

LINQ (Language Integrated Query)

It is a query language, introduced in .NET 3.5 framework. It is used to make the query for data sources with C# or Visual Basics programming languages.

Entity Framework

It is an ORM based open source framework which is used to work with a database using .NET objects. It eliminates a lot of developers effort to handle the database. It is Microsoft's recommended technology to deal with the database.

Parallel LINQ

Parallel LINQ or PLINQ is a parallel implementation of LINQ to objects. It combines the simplicity and readability of LINQ and provides the power of parallel programming.

It can improve and provide fast speed to execute the LINQ query by using all available computer capabilities.

Apart from the above features and libraries, .NET includes other APIs and Model to improve and enhance the .NET framework.

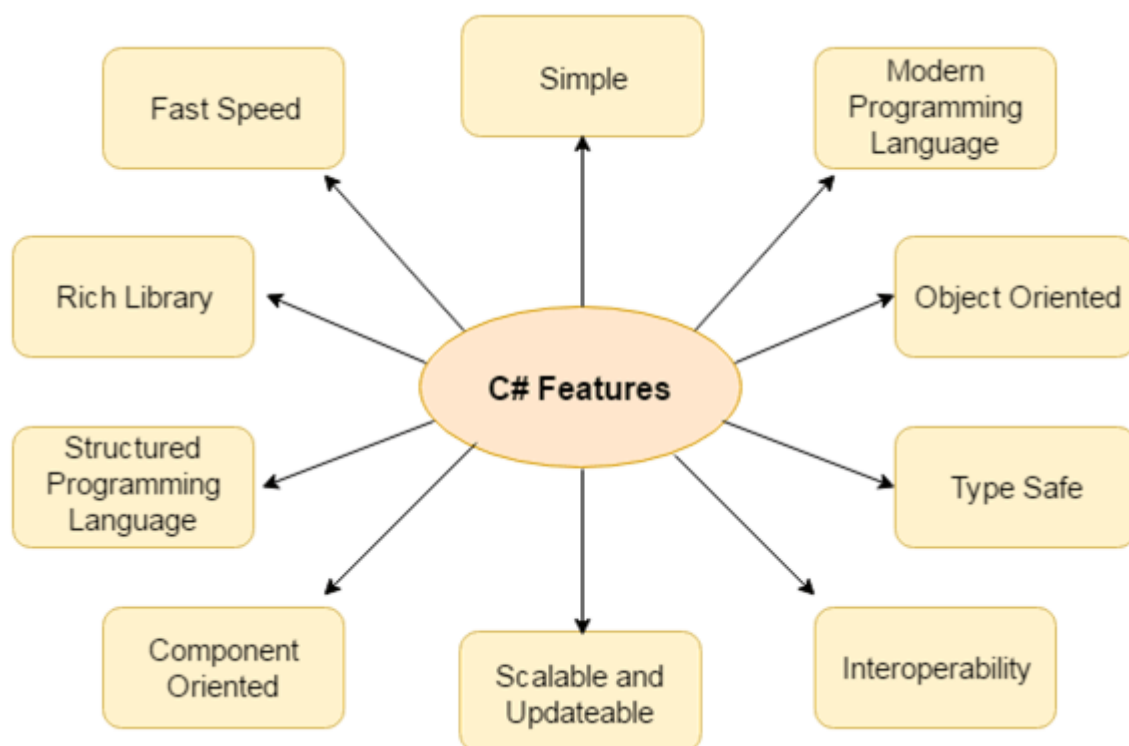
In 2015, Task parallel and Task parallel libraries were added. In .NET 4.5, a task-based asynchronous model was added.

C# Features

C# is object oriented programming language. It provides a lot of **features** that are given below.

1. Simple
2. Modern programming language

3. Object oriented
4. Type safe
5. Interoperability
6. Scalable and Updateable
7. Component oriented
8. Structured programming language
9. Rich Library
10. Fast speed



1) Simple

C# is a simple language in the sense that it provides structured approach (to break the problem into parts), rich set of library functions, data types etc.

2) Modern Programming Language

C# programming is based upon the current trend and it is very powerful and simple for building scalable, interoperable and robust applications.

3) Object Oriented

C# is object oriented programming language. OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grow.

4) Type Safe

C# type safe code can only access the memory location that it has permission to execute. Therefore it improves a security of the program.

5) Interoperability

Interoperability process enables the C# programs to do almost anything that a native C++ application can do.

6) Scalable and Updateable

C# is automatic scalable and updateable programming language. For updating our application we delete the old files and update them with new ones.

7) Component Oriented

C# is component oriented programming language. It is the predominant software development methodology used to develop more robust and highly scalable applications.

8) Structured Programming Language

C# is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify.

9) Rich Library

C# provides a lot of inbuilt functions that makes the development fast.

10) Fast Speed

The compilation and execution time of C# language is fast.

C# - PROGRAM STRUCTURE

Before we study basic building blocks of the C# programming language, let us look at a bare minimum C# program structure so that we can take it as a reference in upcoming chapters.

Creating Hello World Program

A C# program consists of the following parts:

- Namespace declaration
- A class
- Class methods
- Class attributes
- A Main method
- Statements and Expressions
- Comments

Let us look at a simple code that prints the words "Hello World":

```
using System;
namespace HelloWorldApplication
{
    class HelloWorld
    {
        static void Main(string[] args)
        {
            /* my first program in C# */
            Console.WriteLine("Hello World");
            Console.ReadKey();
        }
    }
}
```

When this code is compiled and executed, it produces the following result:

```
Hello World
```

Let us look at the various parts of the given program:

- The first line of the program **using System;** - the **using** keyword is used to include the **System** namespace in the program. A program generally has multiple **using** statements.
- The next line has the **namespace** declaration. A **namespace** is a collection of classes. The *HelloWorldApplication* namespace contains the class *HelloWorld*.
- The next line has a **class** declaration, the class *HelloWorld* contains the data and method definitions that your program uses. Classes generally contain multiple methods. Methods define the behavior of the class. However, the *HelloWorld* class has only one method **Main**.
- The next line defines the **Main** method, which is the **entry point** for all C# programs. The **Main** method states what the class does when executed.
- The next line **/*...*/** is ignored by the compiler and it is put to add **comments** in the program.
- The Main method specifies its behavior with the statement **Console.WriteLine " HelloWorld "** ;
WriteLine is a method of the *Console* class defined in the *System* namespace. This statement

causes the message "Hello, World!" to be displayed on the screen.

- The last line **Console.ReadKey;** is for the VS.NET Users. This makes the program wait for a key press and it prevents the screen from running and closing quickly when the program is launched from Visual Studio .NET.

It is worth to note the following

- points:C# is case sensitive.
- All statements and expression must end with a semicolon ; .
- The program execution starts at the Main method.
- Unlike Java, program file name could be different from the class name.

Compiling and Executing the Program

If you are using Visual Studio.Net for compiling and executing C# programs, take the following steps:

- Start Visual Studio.
- On the menu bar, choose File -> New -> Project.
- Choose Visual C# from templates, and then choose Windows.
- Choose Console Application.
- Specify a name for your project and click OK button.
- This creates a new project in Solution Explorer.
- Write code in the Code Editor.
- Click the Run button or press F5 key to execute the project. A Command Prompt window appears that contains the line Hello World.

You can compile a C# program by using the command-line instead of the Visual Studio IDE:

- Open a text editor and add the above-mentioned code.
- Save the file as **helloworld.cs**
- Open the command prompt tool and go to the directory where you saved the file.
- Type **csc helloworld.cs** and press enter to compile your code.
- If there are no errors in your code, the command prompt takes you to the next line and generates **helloworld.exe** executable file.
- Type **helloworld** to execute your program.

You can see the output Hello World printed on the screen

C# Variable

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

The basic variable type available in C# can be categorized as:

Variable Type	Example
Decimal types	decimal
Boolean types	True or false value, as assigned
Integral types	int, char, byte, short, long
Floating point types	float and double
Nullable types	Nullable data types

Let's see the syntax to declare a variable:

1. `type variable_list;`

The example of declaring variable is given below:

1. `int i, j;`
2. `double d;`
3. `float f;`
4. `char ch;`

Here, i, j, d, f, ch are variables and int, double, float, char are data types.

We can also provide values while declaring the variables as given below:

1. `int i=2,j=4; //declaring 2 variable of integer type`
2. `float f=40.2;`
3. `char ch='B';`

Rules for defining variables

A variable can have alphabets, digits and underscore.

A variable name can start with alphabet and underscore only. It can't start with digit.

No white space is allowed within variable name.

A variable name must not be any reserved word or keyword e.g. char, float etc.

Valid variable names:

1. `int x;`
2. `int _x;`
3. `int k20;`

Invalid variable names:

1. `int 4;`
2. `int x y;`
3. `int double;`

Identifiers:

An identifier is a name given to various program elements, such as variables, methods, classes, namespaces, and more, to identify and reference them within your code. Identifiers are user-defined and should follow specific naming rules and conventions in a programming language. Here are some key points about identifiers:

Naming Rules:

Identifiers must start with a letter or an underscore.

They can be followed by letters, digits, or underscores.

Identifiers are case-sensitive, meaning that "myVar" and "myvar" are considered different identifiers in C#.

Examples:

Variable Identifier: `int age;`

Method Identifier: `void PrintName(string name) { ... }`

Class Identifier: `public class Person { ... }`

Use in Code:

Identifiers are used to reference program elements throughout your code. For example, you use a method identifier to call that method, a variable identifier to store and retrieve values, and a class identifier to define a class's structure.

C# Keywords

A keyword is a reserved word. You cannot use it as a variable name, constant name etc.

In C# keywords cannot be used as identifiers. However, if we want to use the keywords as identifiers, we may prefix the keyword with @ character.

A list of Reserved Keywords available in C# programming language is given below:

abstract	base	as	bool	break	catch	case
byte	char	checked	class	const	continue	decimal
private	protected	public	return	readonly	ref	sbyte
explicit	extern	false	finally	fixed	float	for
foreach	goto	if	implicit	in	in (generic modifier)	int
ulong	ushort	unchecked	using	unsafe	virtual	void
null	object	operator	out	out (generic modifier)	override	params
default	delegate	do	double	else	enum	event
sealed	short	sizeof	stackalloc	static	string	struct
switch	this	throw	true	try	typeof	uint
abstract	base	as	bool	break	catch	case
volatile	while					

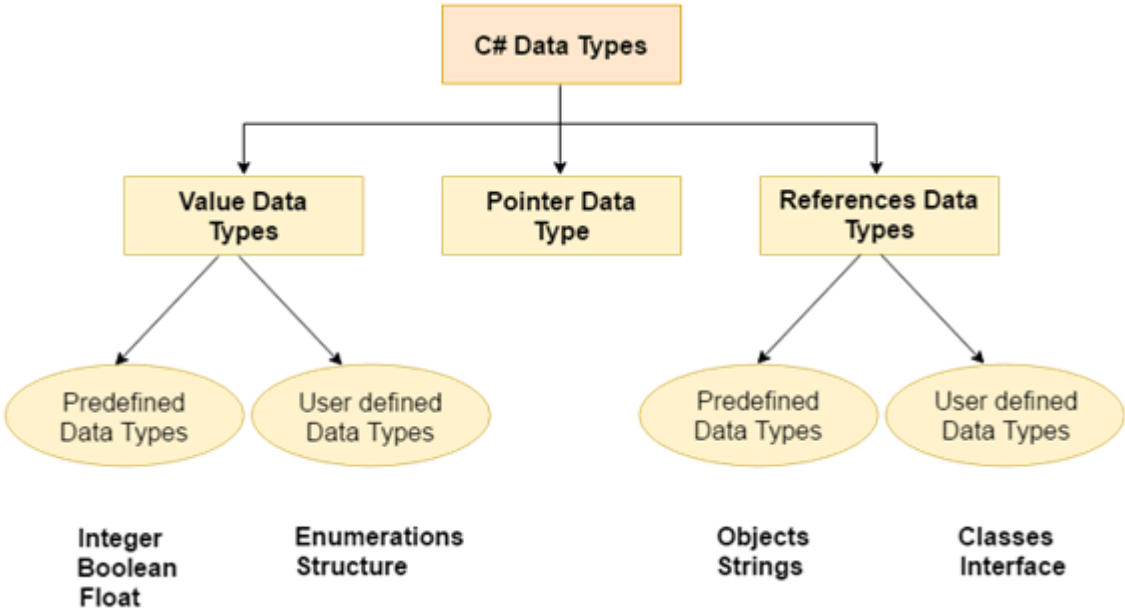
Some identifiers which have special meaning in context of code are called as **Contextual Keywords**.

A list of Contextual Keywords available in C# programming language is given below:

add	group	ascending	descending	dynamic	from	get
global	alias	into	join	let	select	set
partial (type)	partial(method)	remove	orderby			

C# Data Types

A data type specifies the type of data that a variable can store such as integer, floating, character etc.



There are 3 types of data types in C# language.

Types	Data Types
Value Data Type	short, int, char, float, double etc
Reference Data Type	String, Class, Object and Interface
Pointer Data Type	Pointers

Value Data Type

The value data types are integer-based and floating-point based. C# language supports both signed and unsigned literals.

There are 2 types of value data type in C# language.

1) Predefined Data Types - such as Integer, Boolean, Float, etc.

2) User defined Data Types - such as Structure, Enumerations, etc.

The memory size of data types may change according to 32 or 64 bit operating system.

Let's see the value data types. It size is given according to 32 bit OS.

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 127
short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 65,535
int	4 byte	-2,147,483,648 to -2,147,483,647
signed int	4 byte	-2,147,483,648 to -2,147,483,647
unsigned int	4 byte	0 to 4,294,967,295
long	8 byte	?9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
signed long	8 byte	?9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long	8 byte	0 - 18,446,744,073,709,551,615
float	4 byte	$1.5 * 10^{-45}$ - $3.4 * 10^{38}$, 7-digit precision

double	8 byte	$5.0 * 10^{-324}$ - $1.7 * 10^{308}$, 15-digit precision
decimal	16 byte	at least $-7.9 * 10^{28}$ - $7.9 * 10^{28}$, with at least 28-digit precision

Reference Data Type

The reference data types do not contain the actual data stored in a variable, but they contain a reference to the variables.

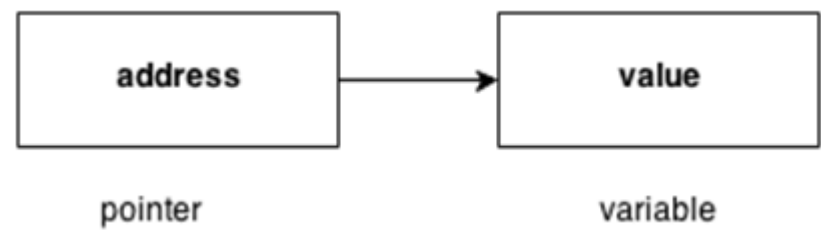
If the data is changed by one of the variables, the other variable automatically reflects this change in value.

There are 2 types of reference data type in C# language.

- 1) **Predefined Types** - such as Objects, String.
- 2) **User defined Types** - such as Classes, Interface.

Pointer Data Type

The pointer in C# language is a variable, it is also known as locator or indicator that points to an address of a value.



Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
* (asterisk sign)	Indirection operator	Access the value of an address.

Declaring a pointer

The pointer in C# language can be declared using * (asterisk symbol).

1. `int * a; //pointer to int`
2. `char * c; //pointer to char`

C# operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise etc.

There are following types of operators to perform different types of operations in C# language.

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Unary Operators
- Ternary Operators
- Misc Operators

	Operator	Type
Binary Operator	+, -, *, /, %	Arithmetic Operators
	<, <=, >, >=, ==, !=	Relational Operators
	&&, , !	Logical Operators
	&, , <<, >>, ~, ^	Bitwise Operators
	=, +=, -=, *=, /=, %=	Assignment Operators
Unary Operator	++, --	Unary Operator
Ternary Operator	?:	Ternary or Conditional Operator

Precedence of Operators in C#

The precedence of operator specifies that which operator will be evaluated first and next. The associativity specifies the operators direction to be evaluated, it may be left to right or right to left.

Let's understand the precedence by the example given below:

1. `int data= 10+ 5*5`

The "data" variable will contain 35 because * (multiplicative operator) is evaluated before + (additive operator).

The precedence and associativity of C# operators is given below:

Category (By Precedence)	Operator(s)	Associativity
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to Left
Additive	+ -	Left to Right
Multiplicative	% / *	Left to Right
Relational	< > <= >=	Left to Right
Shift	<< >>	Left to Right
Equality	== !=	Right to Left
Logical AND	&	Left to Right
Logical OR		Left to Right
Logical XOR	^	Left to Right
Conditional OR		Left to Right
Conditional AND	&&	Left to Right
Null Coalescing	??	Left to Right

Ternary	?:	Right to Left
Assignment	= *= /= %= += -= <<= >>= &= ^= = =>	Right to Left

Unit -2 Control Statements

Control statements, often referred to as control structures or control flow statements, are fundamental elements in programming that determine the order of execution of instructions within a program. They allow you to direct how your program behaves under different conditions and enable you to create complex logic and decision-making processes. Control statements are vital for implementing conditional behavior, loops, and branching within your code

C# if-else

In C# programming, the *if statement* is used to test the condition. There are various types of if statements in C#.

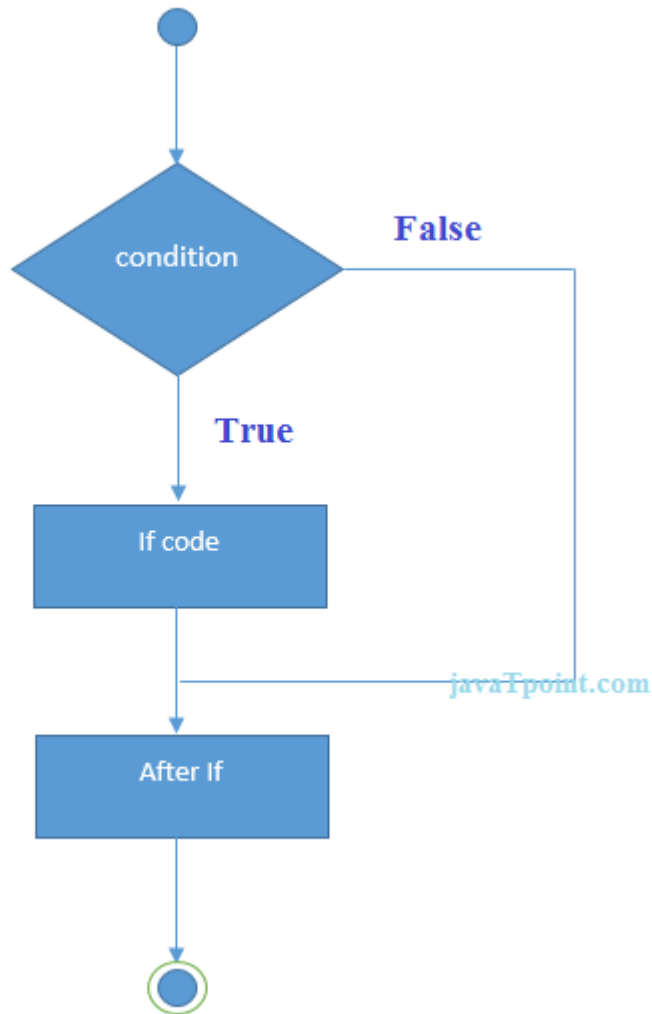
- if statement
- if-else statement
- nested if statement
- if-else-if ladder

C# IF Statement

The C# if statement tests the condition. It is executed if condition is true.

Syntax:

1. **if**(condition){
2. *//code to be executed*
3. }



C# If Example

```
1. using System;
2. public class IfExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int num = 10;
7.         if (num % 2 == 0)
8.         {
9.             Console.WriteLine("It is even number");
10.        }
11.    }
12. }
13. }
```

Output:

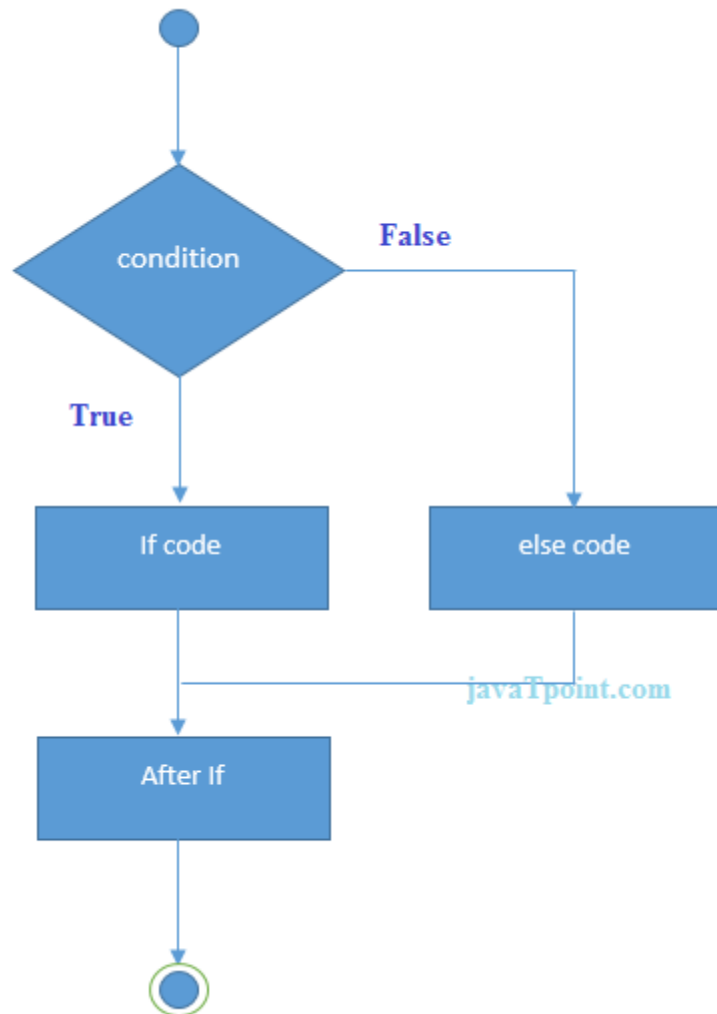
```
It is even number
```

C# IF-else Statement

The C# if-else statement also tests the condition. It executes the *if block* if condition is true otherwise *else block* is executed.

Syntax:

1. **if**(condition){
2. *//code if condition is true*
3. }**else**{
4. *//code if condition is false*
5. }



C# If-else Example

1. using System;
2. **public class** IfExample
3. {
4. **public static void** Main(string[] args)
5. {
6. **int** num = 11;
7. **if** (num % 2 == 0)
8. {
9. Console.WriteLine("It is even number");
10. }

```

11.     else
12.     {
13.         Console.WriteLine("It is odd number");
14.     }
15.
16. }
17. }

```

Output:

```
It is odd number
```

C# If-else Example: with input from user

In this example, we are getting input from the user using **Console.ReadLine()** method. It returns string. For numeric value, you need to convert it into int using **Convert.ToInt32()** method.

```

1. using System;
2. public class IfExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         Console.WriteLine("Enter a number:");
7.         int num = Convert.ToInt32(Console.ReadLine());
8.
9.         if (num % 2 == 0)
10.        {
11.            Console.WriteLine("It is even number");
12.        }
13.        else
14.        {
15.            Console.WriteLine("It is odd number");
16.        }
17.
18.    }
19. }

```

Output:

```
Enter a number:11
It is odd number
```

Output:

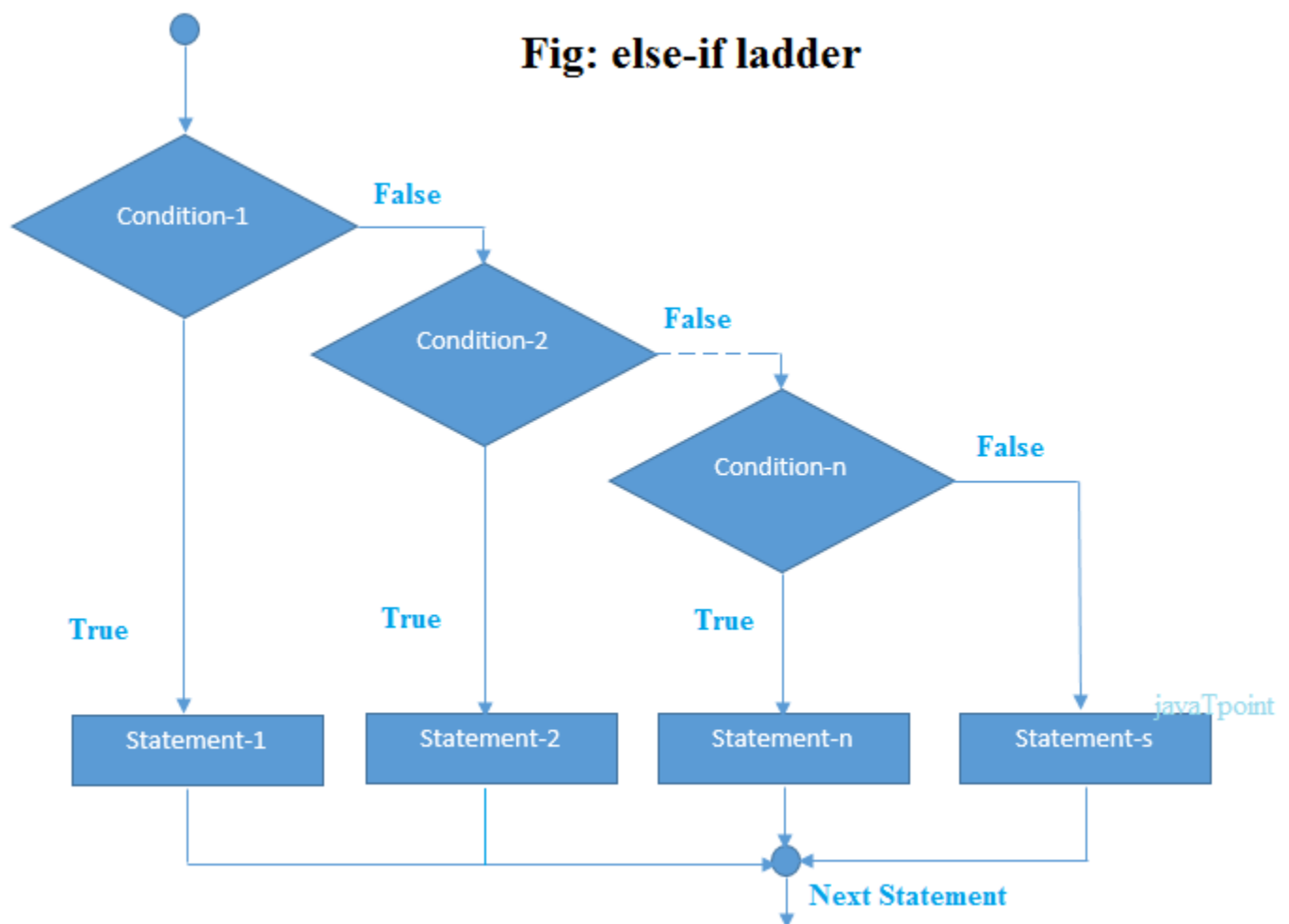
```
Enter a number:12
It is even number
```

C# IF-else-if ladder Statement

The C# if-else-if ladder statement executes one condition from multiple statements.

Syntax:

1. **if**(condition1){
2. //code to be executed if condition1 is true
3. }**else if**(condition2){
4. //code to be executed if condition2 is true
5. }
6. **else if**(condition3){
7. //code to be executed if condition3 is true
8. }
9. ...
10. **else**{
11. //code to be executed if all the conditions are false
12. }



C# If else-if Example

1. using System;
2. **public class** IfExample
3. {
4. **public static void** Main(string[] args)
5. {

```
6. Console.WriteLine("Enter a number to check grade:");
7. int num = Convert.ToInt32(Console.ReadLine());
8.
9. if (num < 0 || num > 100)
10. {
11.     Console.WriteLine("wrong number");
12. }
13. else if (num >= 0 && num < 50){
14.     Console.WriteLine("Fail");
15. }
16. else if (num >= 50 && num < 60)
17. {
18.     Console.WriteLine("D Grade");
19. }
20. else if (num >= 60 && num < 70)
21. {
22.     Console.WriteLine("C Grade");
23. }
24. else if (num >= 70 && num < 80)
25. {
26.     Console.WriteLine("B Grade");
27. }
28. else if (num >= 80 && num < 90)
29. {
30.     Console.WriteLine("A Grade");
31. }
32. else if (num >= 90 && num <= 100)
33. {
34.     Console.WriteLine("A+ Grade");
35. }
36. }
37. }
```

Output:

```
Enter a number to check grade:66
C Grade
```

Output:

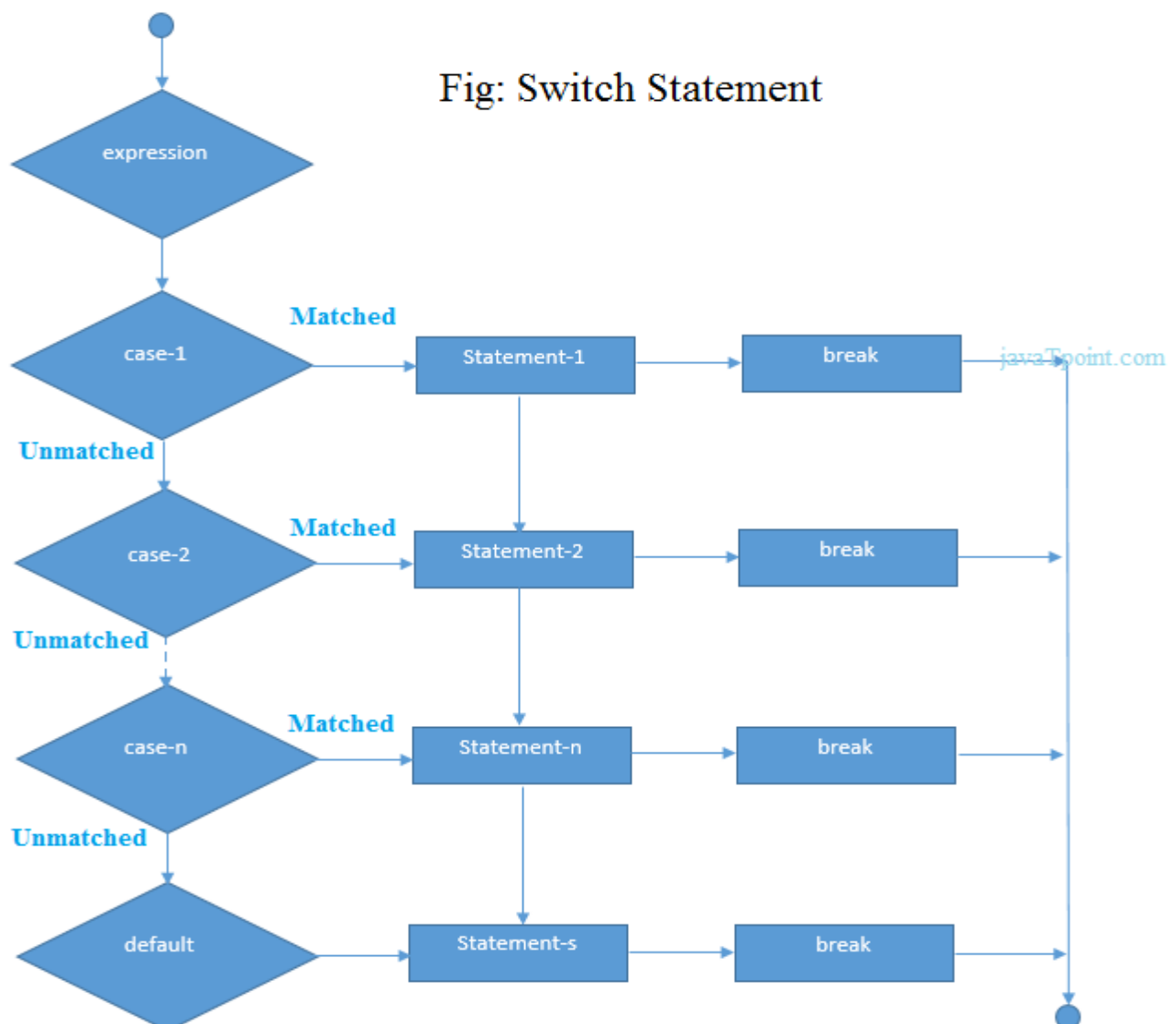
```
Enter a number to check grade:-2
wrong number
```


C# switch

The C# *switch statement* executes one statement from multiple conditions. It is like if-else-if ladder statement in C#.

Syntax:

1. **switch**(expression){
2. **case** value1:
3. //code to be executed;
4. **break**;
5. **case** value2:
6. //code to be executed;
7. **break**;
8.
- 9.
10. **default:**
11. //code to be executed if all cases are not matched;
12. **break**;
13. }



C# Switch Example

```
1. using System;
2. public class SwitchExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         Console.WriteLine("Enter a number:");
7.         int num = Convert.ToInt32(Console.ReadLine());
8.
9.         switch (num)
10.        {
11.            case 10: Console.WriteLine("It is 10"); break;
12.            case 20: Console.WriteLine("It is 20"); break;
13.            case 30: Console.WriteLine("It is 30"); break;
14.            default: Console.WriteLine("Not 10, 20 or 30"); break;
15.        }
16.    }
17. }
```

Output:

```
Enter a number:
10
It is 10
```

Output:

```
Enter a number:
55
Not 10, 20 or 30
```

Note: In C#, break statement is must in switch cases.

C# For Loop

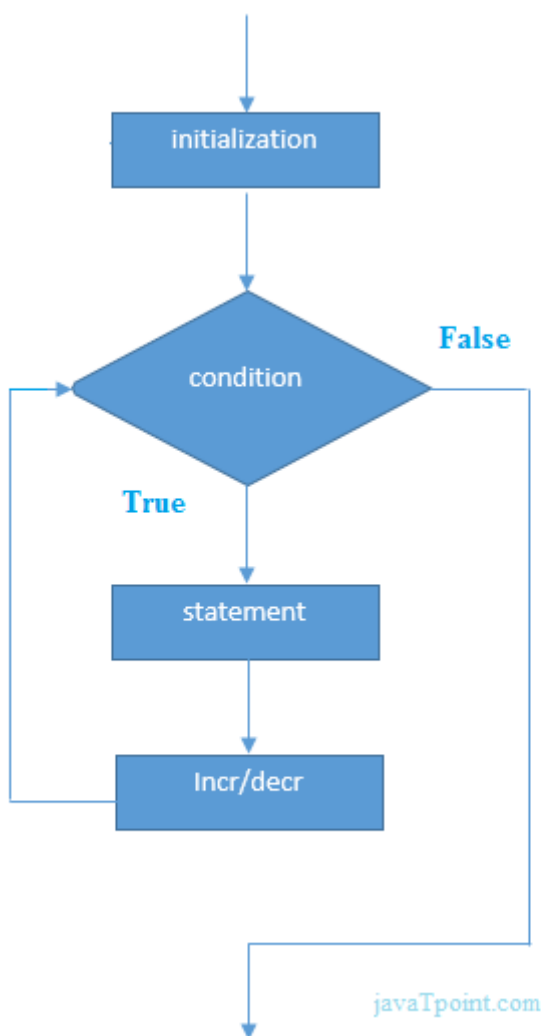
The C# *for loop* is used to iterate a part of the program several times. If the number of iteration is fixed, it is recommended to use for loop than while or do-while loops.

The C# for loop is same as C/C++. We can initialize variable, check condition and increment/decrement value.

Syntax:

```
for(initialization; condition; incr/decr){  
    //code to be executed  
}
```

Flowchart:



C# For Loop Example

1. using System;
2. **public class** ForExample
3. {
4. **public static void** Main(string[] args)
5. {
6. **for**(**int** i=1;i<=10;i++){
7. Console.WriteLine(i);

```
8.     }
9.     }
10. }
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

C# Nested For Loop

In C#, we can use for loop inside another for loop, it is known as nested for loop. The inner loop is executed fully when outer loop is executed one time. So if outer loop and inner loop are executed 3 times, inner loop will be executed 3 times for each outer loop i.e. total 9 times.

Let's see a simple example of nested for loop in C#.

```
1. using System;
2. public class ForExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         for(int i=1;i<=3;i++){
7.             for(int j=1;j<=3;j++){
8.                 Console.WriteLine(i+" "+j);
9.             }
10.        }
11.    }
12. }
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

C# Infinite For Loop

If we use double semicolon in for loop, it will be executed infinite times. Let's see a simple example of infinite for loop in C#.

```
1. using System;
```

```
2. public class ForExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         for (; ; )
7.         {
8.             Console.WriteLine("Infinitive For Loop");
9.         }
10.    }
11. }
```

Output:

```
Infinitive For Loop
Infinitive For Loop
Infinitive For Loop
Infinitive For Loop
Infinitive For Loop
ctrl+c
```

C# Do-While Loop

The C# *do-while loop* is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The C# *do-while loop* is executed at least once because condition is checked after loop body.

Syntax:

```
1. do{
2.     //code to be executed
3. }while(condition);
```

C# do-while Loop Example

Let's see a simple example of C# do-while loop to print the table of 1.

```
1. using System;
2. public class DoWhileExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int i = 1;
7.
8.         do{
9.             Console.WriteLine(i);
```

```
10.         i++;
11.     } while (i <= 10) ;
12.
13. }
14. }
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

C# Nested do-while Loop

In C#, if you use do-while loop inside another do-while loop, it is known as nested do-while loop. The nested do-while loop is executed fully for each outer do-while loop.

Let's see a simple example of nested do-while loop in C#.

```
1. using System;
2. public class DoWhileExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int i=1;
7.
8.         do{
9.             int j = 1;
10.
11.             do{
12.                 Console.WriteLine(i+" "+j);
13.                 j++;
14.             } while (j <= 3) ;
15.             i++;
16.         } while (i <= 3) ;
17.     }
18. }
```

Output:

```
1 1
1 2
1 3
2 1
2 2
```

```
2 3
3 1
3 2
3 3
```

C# Infinitive do-while Loop

In C#, if you pass **true** in the do-while loop, it will be infinitive do-while loop.

Syntax:

```
do{
//code to be executed
}while(true);
```

C# Infinitive do-while Loop Example

```
1. using System;
2. public class WhileExample
3. {
4.     public static void Main(string[] args)
5.     {
6.
7.         do{
8.             Console.WriteLine("Infinitive do-while Loop");
9.         } while(true);
10.    }
11. }
```

Output:

```
Infinitive do-while Loop
Infinitive do-while Loop
Infinitive do-while Loop
Infinitive do-while Loop
Infinitive do-while Loop
ctrl+c
```

C# While Loop

In C#, *while loop* is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop than for loop.

Syntax:

```
while(condition){
//code to be executed
}
```

Flowchart:

C# While Loop Example

Let's see a simple example of while loop to print table of 1.

```
1. using System;
2. public class WhileExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int i=1;
7.         while(i<=10)
8.         {
9.             Console.WriteLine(i);
10.            i++;
11.        }
12.    }
13. }
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

C# Nested While Loop Example:

In C#, we can use while loop inside another while loop, it is known as nested while loop. The nested while loop is executed fully when outer loop is executed once.

Let's see a simple example of nested while loop in C# programming language.

```
1. using System;
2. public class WhileExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int i=1;
7.         while(i<=3)
8.         {
9.             int j = 1;
10.            while (j <= 3)
```



```

11.     {
12.         Console.WriteLine(i+ " "+j);
13.         j++;
14.     }
15.     i++;
16. }
17. }
18. }

```

Output:

```

1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3

```

C# Infinitive While Loop Example:

We can also create infinite while loop by passing *true* as the test condition.

```

1. using System;
2. public class WhileExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         while(true)
7.         {
8.             Console.WriteLine("Infinitive While Loop");
9.         }
10.    }
11. }

```

Output:

```

Infinitive While Loop
Infinitive While Loop
Infinitive While Loop
Infinitive While Loop
Infinitive While Loop
ctrl+c

```

Jump Control Statements:

Jump control statements provide a way to change the flow of program execution by transferring control to another part of the program. The primary jump control statements include:

break Statement: The break statement is used to exit a loop prematurely. It is often used within loops to terminate them before the loop condition is met.

continue Statement: The continue statement is used to skip the current iteration of a loop and continue to the next iteration.

return Statement: The return statement is used to exit a function or method and optionally return a value.

goto Statement: The goto statement is used to transfer control to a labeled statement within the program. It is rarely used and generally discouraged in modern programming.

Control statements are essential for creating complex and dynamic program behavior. By using these statements effectively, you can implement conditional logic, iterate through data, and manage program flow, making your code more powerful and adaptable.

Unit - 3 Arrays

- 3.1 Introduction to array
- 3.2 Declaration and initialization of array
- 3.3 Accessing data from an array
- 3.4 Multi-dimensional arrays
- 3.5 Jagged arrays, Param arrays , and array class.

Introduction to Arrays in C#:

In C#, an array is a collection of elements of the same data type. Arrays are commonly used to store and manipulate multiple values under a single variable name. They provide efficient access to elements and are used in various programming scenarios.

Declaration and Initialization of Arrays in C#:

To declare and initialize an array in C#, you follow these steps:

Declaration:

Start by declaring the array variable. You specify the data type of the elements in the array and the name of the array variable.

`DataType[] ArrayName;`

For example, to declare an array of integers:

```
int[] numbers;
```

Initialization:

After declaring the array, you need to initialize it. Initialization involves allocating memory for the array and specifying its size. There are a few ways to initialize arrays:

a. Using the new Keyword:

You can use the new keyword to create a new array and specify its size. For example, to create an integer array with 5 elements:

```
int[] numbers = new int[5];
```

This initializes the "numbers" array with 5 elements, all set to their default values (0 for integers).

b. Using Array Initializers:

You can also initialize an array with specific values using an array initializer, which is a comma-separated list of values enclosed in curly braces {}.

```
int[] numbers = { 1, 2, 3, 4, 5 };
```

In this example, the "numbers" array is initialized with the specified values.

Accessing Array Elements:

You can access elements of an array using their index, which starts at 0 for the first element, 1 for the second element, and so on.

```
int firstNumber = numbers[0]; // Accesses the first element  
int secondNumber = numbers[1]; // Accesses the second  
element
```

Array Length:

To get the length (the number of elements) of an array, you can use the Length property:

```
int length = numbers.Length; // Gets the length of the  
"numbers" array (5 in this case)
```

Modifying Array Elements:

You can modify array elements by assigning new values to specific indices:

```
numbers[0] = 10; // Modifies the first element to have the value 10
```

Arrays are a powerful and commonly used data structure in C# for working with collections of data.

C# Multidimensional Arrays

The multidimensional array is also known as rectangular arrays in C#. It can be two dimensional or three dimensional. The data is stored in tabular form (row * column) which is also known as matrix.

To create multidimensional array, we need to use comma inside the square brackets. For example:

```
int[,] arr=new int[3,3];//declaration of 2D array
int[,,] arr=new int[3,3,3];//declaration of 3D array
```

C# Multidimensional Array Example

Let's see a simple example of multidimensional array in C# which declares, initializes and traverse two dimensional array.

```
1. using System;
2. public class MultiArrayExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int[,] arr=new int[3,3];//declaration of 2D array
7.         arr[0,1]=10;//initialization
8.         arr[1,2]=20;
9.         arr[2,0]=30;
10.
11.         //traversal
12.         for(int i=0;i<3;i++){
13.             for(int j=0;j<3;j++){
14.                 Console.Write(arr[i,j]+" ");
15.             }
16.             Console.WriteLine();//new line at each row
17.         }
18.     }
19. }
```

Output:

```
0 10 0
0 0 20
30 0 0
```

C# Multidimensional Array Example: Declaration and initialization at same time

There are 3 ways to initialize multidimensional array in C# while declaration.

```
int[,] arr = new int[3,3]= { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
```

We can omit the array size.

```
int[,] arr = new int[,]{{ 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
```

We can omit the new operator also.

```
int[,] arr = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
```

Let's see a simple example of multidimensional array which initializes array at the time of declaration.

```
1. using System;
2. public class MultiArrayExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         int[,] arr = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };//declaration and initialization
7.
8.         //traversal
9.         for(int i=0;i<3;i++){
10.             for(int j=0;j<3;j++){
11.                 Console.Write(arr[i,j]+ " ");
12.             }
13.             Console.WriteLine();//new line at each row
14.         }
15.     }
16. }
```

Output:

```
1 2 3
4 5 6
7 8 9
```

C# Jagged Arrays

In C#, jagged array is also known as "array of arrays" because its elements are arrays. The element size of jagged array can be different.

Declaration of Jagged array

Let's see an example to declare jagged array that has two elements.

```
int[][] arr = new int[2][];
```

Initialization of Jagged array

Let's see an example to initialize jagged array. The size of elements can be different.

```
arr[0] = new int[4];
```

```
arr[1] = new int[6];
```

Initialization and filling elements in Jagged array

Let's see an example to initialize and fill elements in jagged array.

```
arr[0] = new int[4] { 11, 21, 56, 78 };  
arr[1] = new int[6] { 42, 61, 37, 41, 59, 63 };
```

Here, size of elements in jagged array is optional. So, you can write above code as given below:

```
arr[0] = new int[] { 11, 21, 56, 78 };  
arr[1] = new int[] { 42, 61, 37, 41, 59, 63 };
```

C# Jagged Array Example

Let's see a simple example of jagged array in C# which declares, initializes and traverse jagged arrays.

```
1. public class JaggedArrayTest  
2. {  
3.     public static void Main()  
4.     {  
5.         int[][] arr = new int[2][]; // Declare the array  
6.  
7.         arr[0] = new int[] { 11, 21, 56, 78 }; // Initialize the array  
8.         arr[1] = new int[] { 42, 61, 37, 41, 59, 63 };  
9.  
10.        // Traverse array elements  
11.        for (int i = 0; i < arr.Length; i++)  
12.        {  
13.            for (int j = 0; j < arr[i].Length; j++)  
14.            {  
15.                System.Console.Write(arr[i][j] + " ");  
16.            }  
17.            System.Console.WriteLine();  
18.        }  
19.    }  
20. }
```

Output:

```
11 21 56 78  
42 61 37 41 59 63
```

Initialization of Jagged array upon Declaration

Let's see an example to initialize the jagged array while declaration.

```
1. int[][] arr = new int[3][]{
```


2. `new int[] { 11, 21, 56, 78 },`
3. `new int[] { 2, 5, 6, 7, 98, 5 },`
4. `new int[] { 2, 5 }`
5. `};`

C# Params

In C#, params is a keyword which is used to specify a parameter that takes variable number of arguments. It is useful when we don't know the number of arguments prior. Only one params keyword is allowed and no additional parameter is permitted after params keyword in a function declaration.

C# Params Example 1

```
using System;

namespace AccessSpecifiers
{
    class Program
    {
        // User defined function
        public void Show(params int[] val) // Params Paramater
        {
            for (int i=0; i<val.Length; i++)
            {
                Console.WriteLine(val[i]);
            }
        }

        // Main function, execution entry point of the program
        static void Main(string[] args)
        {
            Program program = new Program(); // Creating Object

            program.Show(2,4,6,8,10,12,14); // Passing arguments of variable length
        }
    }
}
```

```
}  
  
}
```

Output:

```
2  
4  
6  
8  
10  
12  
14
```

C# Array class

C# provides an Array class to deal with array related operations. It provides methods for creating, manipulating, searching, and sorting elements of an array. This class works as the base class for all arrays in the .NET programming environment.

C# Array class Signature

[SerializableAttribute]

[ComVisibleAttribute(**true**)]

public abstract class Array : ICloneable, IList, ICollection,
IEnumerable, IStructuralComparable, IStructuralEquatable

Note: In C#, Array is not part of collection but considered as collection because it is based on the IList interface.

C# Array Properties

Property	Description
IsFixedSize	It is used to get a value indicating whether the Array has a fixed size or not.
IsReadOnly	It is used to check that the Array is read-only or not.
IsSynchronized	It is used to check that access to the Array is synchronized or not.
Length	It is used to get the total number of elements in all the dimensions of the Array.
LongLength	It is used to get a 64-bit integer that represents the total number of elements in all the dimensions of the Array.
Rank	It is used to get the rank (number of dimensions) of the Array.
SyncRoot	It is used to get an object that can be used to synchronize access to the Array.

C# Array Methods

Method	Description
AsReadOnly<T>(T[])	It returns a read-only wrapper for the specified array.
BinarySearch(Array,Int32,Int32,Object)	It is used to search a range of elements in a one-dimensional sorted array for a value.
BinarySearch(Array,Object)	It is used to search an entire one-dimensional sorted array for a specific element.
Clear(Array,Int32,Int32)	It is used to set a range of elements in an array to the default value.
Clone()	It is used to create a shallow copy of the Array.
Copy(Array,Array,Int32)	It is used to copy elements of an array into another array by specifying starting index.
CopyTo(Array,Int32)	It copies all the elements of the current one-dimensional array to the specified one-dimensional array starting at the specified destination array index
CreateInstance(Type,Int32)	It is used to create a one-dimensional Array of the specified Type and length.
Empty<T>()	It is used to return an empty array.
Finalize()	It is used to free resources and perform cleanup operations.
Find<T>(T[],Predicate<T>)	It is used to search for an element that matches the conditions defined by the specified predicate.
IndexOf(Array,Object)	It is used to search for the specified object and returns the index of its first occurrence in a one-dimensional array.
Initialize()	It is used to initialize every element of the value-type Array by calling the default constructor of the value type.
Reverse(Array)	It is used to reverse the sequence of the elements in the

	entire one-dimensional Array.
Sort(Array)	It is used to sort the elements in an entire one-dimensional Array.
ToString()	It is used to return a string that represents the current object.

C# Array Example

```

1. using System;
2. namespace CSharpProgram
3. {
4.     class Program
5.     {
6.         static void Main(string[] args)
7.         {
8.             // Creating an array
9.             int[] arr = new int[6] { 5, 8, 9, 25, 0, 7 };
10.            // Creating an empty array
11.            int[] arr2 = new int[6];
12.            // Displaying length of array
13.            Console.WriteLine("length of first array: "+arr.Length);
14.            // Sorting array
15.            Array.Sort(arr);
16.            Console.Write("First array elements: ");
17.            // Displaying sorted array
18.            PrintArray(arr);
19.            // Finding index of an array element
20.            Console.WriteLine("\nIndex position of 25 is "+Array.IndexOf(arr,25));
21.            // Coping first array to empty array
22.            Array.Copy(arr, arr2, arr.Length);
23.            Console.Write("Second array elements: ");
24.            // Displaying second array
25.            PrintArray(arr2);
26.            Array.Reverse(arr);
27.            Console.Write("\nFirst Array elements in reverse order: ");
28.            PrintArray(arr);
29.        }
30.        // User defined method for iterating array elements
31.        static void PrintArray(int[] arr)
32.        {
33.            foreach (Object elem in arr)
34.            {

```

```
35.         Console.Write(elem+" ");
36.     }
37. }
38. }
```

Output:

```
length of first array: 6
First array elements: 0 5 7 8 9 25
Index position of 25 is 5
Second array elements: 0 5 7 8 9 25
First Array elements in reverse order: 25 9 8 7 5 0
```

Unit - 4 Strings

4.1 Introduction to Strings

4.2 Creating an String Object

4.3 Methods of String Class

4.4 String Functions

C# Strings

In C#, string is an object of **System.String** class that represent sequence of characters. We can perform many operations on strings such as concatenation, comparision, getting substring, search, trim, replacement etc.

string vs String

In C#, *string* is keyword which is an alias for *System.String* class. That is why string and String are equivalent. We are free to use any naming convention.

```
string s1 = "hello";//creating string using string keyword
String s2 = "welcome";//creating string using String class
```

C# String Example

```
1. using System;
2. public class StringExample
3. {
4.     public static void Main(string[] args)
5.     {
6.         string s1 = "hello";
7.
8.         char[] ch = { 'c', 's', 'h', 'a', 'r', 'p' };
9.         string s2 = new string(ch);
10.
11.         Console.WriteLine(s1);
```

```
12.     Console.WriteLine(s2);
13. }
14. }
```

Output:

```
hello
csharp
```

How to create Sting objects ?

n C#, you can create a string object in several ways. Here are the most common methods for creating a string object:

1. Using String Literals:

You can create a string object by simply enclosing a sequence of characters in double quotes. This is the most common and straightforward way to create strings in C#.

```
string greeting = "Hello, C#!";
```

2. Using String Constructor:

You can use the String constructor to create a string object from a character array or by repeating a character a specified number of times. For example:

Creating a string from a character array:

```
char[] charArray = { 'H', 'e', 'l', 'l', 'o' };
string greeting = new string(charArray);
```

Creating a string by repeating a character:

```
string stars = new string('*', 5); // Creates a string "*****"
```

3. Using String Concatenation:

You can create a string object by concatenating other strings using the + operator or the String.Concat method.

```
string firstName = "John";  
string lastName = "Doe";  
string fullName = firstName + " " + lastName;
```

4. Using String Interpolation:

String interpolation allows you to create a string by embedding expressions and variables within a string literal.

```
int age = 30;  
string message = $"My age is {age}.";
```

5. Using String Methods:

Some methods in the String class return new string objects. For example, the Substring method creates a substring from an existing string.

```
string original = "Hello, World!";  
string substring = original.Substring(0, 5); // Creates a new  
string "Hello"
```


6. Using String Builder:

When you need to build a string by appending multiple pieces of text, using a `StringBuilder` is more efficient than string concatenation. A `StringBuilder` is mutable, so you can efficiently build a string over time.

```
StringBuilder builder = new StringBuilder();  
builder.Append("Hello");  
builder.Append(", ");  
builder.Append("C#!");  
string result = builder.ToString();
```

These are the common ways to create string objects in C#. You can choose the method that best fits your specific use case, whether it's creating simple string literals, building complex strings, or manipulating existing strings.

C# String methods (Methods of String Class)

Method Name	Description
Clone()	It is used to return a reference to this instance of String.
Compare(String, String)	It is used to compares two specified String objects. It returns an integer that indicates their relative position in the sort order.
CompareOrdinal(String, String)	It is used to compare two specified String objects by evaluating the numeric values of the corresponding Char objects in each string..
CompareTo(String)	It is used to compare this instance with a specified String object. It

	indicates whether this instance precedes, follows, or appears in the same position in the sort order as the specified string.
<u>Concat(String, String)</u>	It is used to concatenate two specified instances of String.
<u>Contains(String)</u>	It is used to return a value indicating whether a specified substring occurs within this string.
<u>Copy(String)</u>	It is used to create a new instance of String with the same value as a specified String.
<u>CopyTo(Int32, Char[], Int32, Int32)</u>	It is used to copy a specified number of characters from a specified position in this instance to a specified position in an array of Unicode characters.
<u>EndsWith(String)</u>	It is used to check that the end of this string instance matches the specified string.
<u>Equals(String, String)</u>	It is used to determine that two specified String objects have the same value.
<u>Format(String, Object)</u>	It is used to replace one or more format items in a specified string with the string representation of a specified object.
<u>GetEnumerator()</u>	It is used to retrieve an object that can iterate through the individual characters in this string.
<u>GetHashCode()</u>	It returns the hash code for this string.
<u>GetType()</u>	It is used to get the Type of the current instance.
<u>GetTypeCode()</u>	It is used to return the TypeCode for class String.
<u>IndexOf(String)</u>	It is used to report the zero-based index of the first occurrence of the specified string in this instance.
<u>Insert(Int32, String)</u>	It is used to return a new string in which a specified string is inserted at a specified index position.
<u>Intern(String)</u>	It is used to retrieve the system's reference to the specified String.

<u>IsInterned(String)</u>	It is used to retrieve a reference to a specified String.
<u>IsNormalized()</u>	It is used to indicate that this string is in Unicode normalization form C.
<u>IsNullOrEmpty(String)</u>	It is used to indicate that the specified string is null or an Empty string.
<u>IsNullOrWhiteSpace(String)</u>	It is used to indicate whether a specified string is null , empty, or consists only of white-space characters.
<u>Join(String, String[])</u>	It is used to concatenate all the elements of a string array, using the specified separator between each element.
<u>LastIndexOf(Char)</u>	It is used to report the zero-based index position of the last occurrence of a specified character within String.
<u>LastIndexOfAny(Char[])</u>	It is used to report the zero-based index position of the last occurrence in this instance of one or more characters specified in a Unicode array.
<u>Normalize()</u>	It is used to return a new string whose textual value is the same as this string, but whose binary representation is in Unicode normalization form C.
<u>PadLeft(Int32)</u>	It is used to return a new string that right-aligns the characters in this instance by padding them with spaces on the left.
<u>PadRight(Int32)</u>	It is used to return a new string that left-aligns the characters in this string by padding them with spaces on the right.
<u>Remove(Int32)</u>	It is used to return a new string in which all the characters in the current instance, beginning at a specified position and continuing through the last position, have been deleted.
<u>Replace(String, String)</u>	It is used to return a new string in which all occurrences of a specified string in the current instance are replaced with another specified string.
<u>Split(Char[])</u>	It is used to split a string into substrings that are based on the

	characters in an array.
<u>StartsWith(String)</u>	It is used to check whether the beginning of this string instance matches the specified string.
<u>Substring(Int32)</u>	It is used to retrieve a substring from this instance. The substring starts at a specified character position and continues to the end of the string.
<u>ToCharArray()</u>	It is used to copy the characters in this instance to a Unicode character array.
<u>ToLower()</u>	It is used to convert String into lowercase.
<u>ToLowerInvariant()</u>	It is used to return convert String into lowercase using the casing rules of the invariant culture.
<u>ToString()</u>	It is used to return instance of String.
<u>ToUpper()</u>	It is used to convert String into uppercase.
<u>Trim()</u>	It is used to remove all leading and trailing white-space characters from the current String object.
<u>TrimEnd(Char[])</u>	It Is used to remove all trailing occurrences of a set of characters specified in an array from the current String object.
<u>TrimStart(Char[])</u>	It is used to remove all leading occurrences of a set of characters specified in an array from the current String object.

String Functions

C# provides a rich set of string functions and methods through the System.String class for manipulating and working with strings. These methods allow you to perform a wide range of operations on strings, from basic tasks like

finding substrings to more advanced operations such as formatting, replacing, and comparing strings.

Here are some of the most commonly used string functions in C#:

1. Length:

Length is a property that returns the number of characters in a string.

```
string text = "Hello, World!";  
int length = text.Length; // Returns 13
```

2. Concatenation:

You can concatenate strings using the + operator or the String.Concat method.

```
string firstName = "John";  
string lastName = "Doe";  
string fullName = firstName + " " + lastName;  
// or  
string fullNameConcat = string.Concat(firstName, " ",  
lastName);
```

3. Substring:

The Substring method is used to extract a portion of a string.

```
string text = "Hello, World!";  
string substring = text.Substring(0, 5); // Returns "Hello"
```

4. ToUpper and ToLower:

ToUpper and ToLower methods change the case of all characters in a string.

```
string text = "Hello, World!";  
string upper = text.ToUpper(); // Converts to uppercase  
string lower = text.ToLower(); // Converts to lowercase
```

5. Replace:

The Replace method replaces all occurrences of a specified substring with another substring.

```
string text = "I like cats, but cats can be challenging.";  
string replaced = text.Replace("cats", "dogs");
```

6. Split:

The Split method divides a string into an array of substrings based on a specified delimiter.

```
string names = "John, Jane, Alice, Bob";  
string[] nameArray = names.Split(','); // Splits into an array  
of names
```

7. Trim:

The Trim, TrimStart, and TrimEnd methods remove leading and trailing whitespace characters from a string.

```
string text = " This is a string with extra spaces. ";  
string trimmed = text.Trim(); // Removes leading and  
trailing spaces
```

8. IndexOf and LastIndexOf:

IndexOf and LastIndexOf methods return the index of the first or last occurrence of a specified substring within the string.

```
string text = "The quick brown fox jumps over the lazy  
dog.";  
int firstIndex = text.IndexOf("fox"); // Returns the index of  
"fox" (16)  
int lastIndex = text.LastIndexOf("the"); // Returns the last  
index of "the" (45)
```

9. StartsWith and EndsWith:

StartsWith and EndsWith methods check whether a string starts or ends with a specified substring.

```
string text = "Hello, World!";  
bool startsWithHello = text.StartsWith("Hello"); // True  
bool endsWithWorld = text.EndsWith("World!"); // True
```

10. Compare and Equals:

The Compare and Equals methods are used for comparing two strings. Compare returns an integer indicating the relative order of the two strings, and Equals checks for equality.

```
string text1 = "apple";  
string text2 = "banana";  
int comparison = string.Compare(text1, text2); // Returns a  
negative value  
bool areEqual = string.Equals(text1, text2); // False
```

These are just a few of the many string functions available in C#. Depending on your specific requirements, you can use these methods to perform operations on strings effectively and efficiently.

Unit - 5 Structures

5.1 Introduction to Structure

5.2 Defining a Structure

5.3 Features of C# Structure

5.4 Class vs. Structure

Introduction to Structures in C#:

In C#, a structure, often referred to as a "struct," is a composite data type that groups together variables of different data types under a single name. Structures allow you to define your own data types with specific attributes, similar to classes. However, structures are value types, meaning that they are stored in the stack memory, and they exhibit different behaviors compared to reference types like classes. Here are key points about structures in C#:

Defining a Structure:

You can define a structure in C# using the struct keyword. The general syntax for defining a structure is as follows:

```
struct StructureName
{
    // Fields or data members
```

```
// Constructor(s)
// Methods (optional)
}
```

Features of Structures:

1. Value Type:

Structures are value types, which means that their instances are stored directly in the memory location where they are declared. They are passed by value, making copies when assigned to other variables or passed as method arguments.

2. Lightweight:

Structures are typically lightweight and use less memory compared to classes. They are suitable for small, simple data structures.

3. Stack Allocation:

Structure instances are allocated on the stack rather than the heap. This leads to faster memory allocation and deallocation.

4. No Inheritance:

Structures cannot be used as a base for inheritance. They do not support inheritance of members.

5. Parameterless Constructor:

By default, a parameterless constructor is provided for structures. You can also define custom constructors if needed.

6. Immutable:

Fields within a structure cannot be modified after the structure is created unless the structure is explicitly marked as mutable using the `mutable` keyword (C# 9.0 and later).

7. Value Semantics:

Structures exhibit value semantics, meaning that changes to one instance of a structure do not affect other instances, and they are compared based on their field values.

8. Nullable:

Structures can be used to create nullable value types by adding a `?` after the structure name. For example, `int?` is a nullable integer.

Feature	Class	Structure (Struct)
Type	Reference Type	Value Type
Memory Allocation	On the Heap	On the Stack
Constructor	Default Constructor Provided	Default Constructor Provided
Custom Constructor	Supported	Supported
Inheritance	Supported	Not Supported
Mutable Fields	Yes (Fields can be modified)	Yes (Fields can be modified)*
Use Case	Complex Data, Objects	Lightweight Data, Simple Types
Performance	Slower in Memory Allocation	Faster in Memo

Unit - 6 Pointers

Pointers are a fundamental concept in C and C++ programming, allowing you to manipulate memory directly and work with addresses of variables. However, in C#, which is designed with a strong focus on safety and managed memory, direct pointer manipulation is not a common or recommended practice. Pointers are mostly used in C# in unsafe code blocks, and their use is restricted for security and stability reasons.

Using Pointers in C#

To use pointers in C#, you must first declare an unsafe context. This is done using the `unsafe` keyword. Once you have declared an unsafe context, you can use the `&` operator to get the address of a variable and the `*` operator to dereference a pointer and access the data it points to.

For example, the following code declares an unsafe context and then uses the `&` operator to get the address of an integer variable:

```
unsafe
{
    int myInt = 10;
    int* pointerToInt = &myInt;
}
```

The `pointerToInt` variable now contains the address of the `myInt` variable.

To access the data that the `pointerToInt` variable points to, you can use the `*` operator. For example, the following code uses the `*` operator to print the value of the `myInt` variable to the console:

```
unsafe
{
    int myInt = 10;
    int* pointerToInt = &myInt;

    Console.WriteLine(*pointerToInt);
}
```

This will print the number 10 to the console.

When to Use Pointers

Pointers can be useful for certain tasks, such as optimizing performance or interfacing with unmanaged code. However, pointers should be used with caution, as they can lead to errors such as memory leaks and buffer overflows.

Here are some examples of when you might want to use pointers in C#:

- To optimize performance for certain tasks, such as sorting a large array or copying a large block of data.
- To interface with unmanaged code, such as libraries written in C or C++.
- To implement custom data structures, such as linked lists and trees.

Pointers in C# are a powerful but low-level feature that allows direct memory manipulation. While they can be useful in certain scenarios, they come with both advantages and disadvantages:

Advantages of Pointers in C#:

1. Low-Level Memory Access:

Pointers provide direct access to memory locations, allowing you to work with memory addresses and perform low-level operations, such as memory copying and manipulation. This can be valuable when working with hardware or interfacing with unmanaged code.

2. Performance Optimization:

In situations where performance is critical, such as in high-performance computing or real-time systems, pointers can be used to optimize memory access and data manipulation.

3. Interop with Unmanaged Code:

When interacting with libraries or components written in languages like C or C++, pointers can be essential for marshaling data and invoking functions that expect pointer parameters.

4. Array and Buffer Manipulation:

Pointers can be used to efficiently manipulate large data buffers, such as reading or writing data from/to files or network streams.

Disadvantages of Pointers in C#:

1. Security Risks:

Pointers can introduce security vulnerabilities if not used with extreme caution. Improper handling of pointers can lead to memory corruption, buffer overflows, and other security issues. This is a significant disadvantage, and C# is designed to minimize these risks.

2. Complexity and Maintenance:

Code that uses pointers can be challenging to understand, debug, and maintain. It may be error-prone and lead to subtle bugs that are difficult to track down.

3. Memory Management:

Pointers do not benefit from C#'s automatic memory management provided by the garbage collector. This means you need to handle memory allocation, deallocation, and object lifetimes manually, which is prone to errors.

4. Lack of Type Safety:

Using pointers can lead to type-related errors, such as invalid type casting or dereferencing the wrong data type, which can lead to unpredictable program behavior.

5. Limited Use Cases:

Pointers are rarely necessary in typical C# application development. Most applications can be written entirely without using pointers, leveraging the safety and productivity features provided by C#.

6. Platform Dependencies:

Code using pointers might not be platform-independent, as it often relies on low-level memory layouts and is sensitive to architecture-specific details.

7. Compiler Restrictions:

C# restricts the use of pointers and unsafe code to maintain program stability and security. This means that using pointers requires enabling the "unsafe" keyword, which may not be allowed in all project settings.

Retrieving the data value using a pointer

In C#, you can retrieve the data value using a pointer by dereferencing the pointer. Dereferencing means accessing the value stored at the memory address pointed to by the pointer. To retrieve the data value using a pointer, follow these steps within an "unsafe" code block:

Declare a pointer variable of the appropriate data type.

Assign the memory address of the variable you want to access to the pointer.

Dereference the pointer to retrieve the data value.

Here's a step-by-step example of how to retrieve a data value using a pointer:

```
using System;
class Program
{
    static unsafe void Main()
    {
        int number = 42; // Create an integer variable
```



```
// Declare a pointer to an integer and assign the address of
'number'
int* ptr = &number;

// Dereference the pointer to retrieve the data value
int retrievedValue = *ptr;

Console.WriteLine($"Original value: {number}");
Console.WriteLine($"Retrieved value: {retrievedValue}");

// Modifying the value through the pointer
*ptr = 100;

Console.WriteLine($"Modified value: {number}");
}
}
```

In this example:

We declare an integer variable `number` and initialize it with the value 42.

Within an "unsafe" code block, we declare a pointer `ptr` to an integer and assign it the address of the `number` variable using the address-of operator `&`.

To retrieve the data value, we dereference the pointer by using the `*` operator and assign the value to the `retrievedValue` variable.

We print the original value of `number` and the value retrieved using the pointer. Both values should be the same.

We modify the value of number through the pointer by dereferencing ptr and setting it to 100. After the modification, the value of number is changed as well.

Please note that using pointers in C# requires enabling the "unsafe" keyword in your code and adjusting your project settings to allow unsafe code. It's important to handle pointers with great care to prevent memory-related issues and security vulnerabilities. Pointers are rarely needed in typical C# application development and should be used only when there is a compelling reason to work at such a low level of memory manipulation.

Passing Pointers as parameter to methods

In C#, you can pass pointers as parameters to methods, allowing you to modify the values pointed to by the pointers within the method. This is a common practice when working with low-level operations or when you need to pass data by reference. To pass pointers as method parameters, you need to define the method with the unsafe keyword and specify the pointer type as the parameter type.

Here's how you can pass pointers as parameters to methods in C#:

- 1. Define the Method with the unsafe Keyword:*

To enable the use of pointers within the method, you must declare the method as unsafe.

2. Specify the Pointer Type as a Parameter:

Define a parameter of the pointer type you want to pass as a parameter. This can be an integer pointer (int*), a character pointer (char*), or any other pointer type.

3. Dereference the Pointer Within the Method:

Inside the method, you can dereference the pointer to access or modify the value pointed to.

Here's an example of passing an integer pointer as a parameter to a method:

```
using System;
class Program
{
    static unsafe void ModifyValue(int* ptr)
    {
        // Dereference the pointer and modify the value
        *ptr = 100;
    }
    static void Main()
    {
        int number = 42;
        int* ptr = &number;
        Console.WriteLine($"Original value: {number}");
        // Call the method and pass the pointer as a parameter
        ModifyValue(ptr);

        Console.WriteLine($"Modified value: {number}");
    }
}
```

```
}  
}
```

In this example:

We declare a method called `ModifyValue`, which takes an integer pointer `int* ptr` as a parameter. The method is marked as `unsafe`.

Inside the method, we dereference the pointer with `*ptr` and set its value to 100.

In the `Main` method, we create an integer variable `number`, declare a pointer `ptr` to it, and initialize the pointer with the address of `number`.

We print the original value of `number`, call the `ModifyValue` method, and pass the pointer `ptr` as a parameter. After the method call, the value of `number` is modified to 100.

6.5 Accessing array elements using a pointer

In C#, you can access array elements using pointers by declaring a pointer to the array and then using pointer arithmetic to navigate through the elements. To do this, you need to mark your code as "unsafe" and use the pointer type that matches the data type of the array elements. Here's a step-by-step guide on how to access array elements using pointers in C#:

1. Declare an Array:

Start by declaring an array of the desired data type. For example, an integer array:

```
int[] numbers = { 10, 20, 30, 40, 50 };
```

2. Mark Code as "Unsafe":

To use pointers, you must define your code within an "unsafe" code block, and you need to enable the "unsafe" keyword in your project settings.

```
unsafe  
{  
    // Unsafe code goes here  
}
```

3. Declare a Pointer:

Declare a pointer variable of the same data type as the array elements. In this case, it's an `int*` (integer pointer) because we're working with an integer array:

```
int* ptr;
```

4. Assign the Array Address to the Pointer:

Assign the address of the array to the pointer using the address-of operator `&`:

```
ptr = numbers;
```

5. Access Array Elements Using Pointers:

You can now use pointer arithmetic to access array elements. For example, to access the first element, use `*ptr`, and to move to the next element, increment the pointer:

```
int firstElement = *ptr; // Access the first element
ptr++; // Move to the next element
int secondElement = *ptr; // Access the second element
```

6. Iterate Through Array:

To iterate through all array elements, you can use a loop:

```
int* ptr = numbers; // Reset the pointer to the beginning
for (int i = 0; i < numbers.Length; i++)
{
    int element = *ptr; // Access each element using the
    pointer
    // Process the element
    ptr++; // Move to the next element
}
```

7. End the Unsafe Block:

After you've finished working with pointers, make sure to end the "unsafe" code block.

Here's a complete example:

```
using System;
class Program
{
    static unsafe void Main()
    {
        int[] numbers = { 10, 20, 30, 40, 50 };
        // Declare a pointer and assign the array's address
        int* ptr = numbers;
        // Access array elements using the pointer
        int firstElement = *ptr;
        ptr++;
        int secondElement = *ptr;

        Console.WriteLine($"First Element: {firstElement}");
        Console.WriteLine($"Second Element: {secondElement}");
    }
}
```

Please note that working with pointers, especially when accessing array elements, requires caution and is not recommended for typical C# application development. It is primarily used in scenarios where you need low-level memory manipulation or interoperation with unmanaged code. Additionally, enabling unsafe code in your project settings is necessary for using pointers.

Unit - 7 Working with Database

7.1 Need of Database in C#

7.2 Database environment setup

7.3 Connecting a C# program with database

7.4 Reading and writing elements to and from the database

A database is a collection of organized data that is stored and managed by a computer system. Databases are used to store a wide variety of data, such as customer records, product information, and financial data.

Databases are important for C# applications because they allow developers to store and manage large amounts of data efficiently. Databases also provide a number of features that can be useful for C# applications, such as:

- **Persistence:** Databases allow developers to store data that persists even after the application is closed. This is important for applications that need to store data that needs to be accessed later, such as customer records or product information.
- **Scalability:** Databases can be scaled to store and manage large amounts of data. This is important for applications that need to store data for a large number of users or that need to store a large amount of data per user.
- **Security:** Databases provide a number of security features that can help to protect data from unauthorized access. This is important for applications that need to store sensitive data, such as customer financial information.

Database Environment Setup

To set up a database environment for C#, you will need to install a database server and a database client. The database server is the software that stores and manages the database. The database client is the software that allows applications to interact with the database server.

There are a number of different database servers and database clients available. Some popular database servers include:

- Microsoft SQL Server
- MySQL
- PostgreSQL
- Oracle Database

Some popular database clients include:

- Entity Framework Core
- ADO.NET
- NHibernate

Once you have installed a database server and a database client, you can create a database and start storing data

Fundamentals of Database connectivity

C# and .Net can work with a majority of databases, the most common being Oracle and Microsoft SQL Server. But with every database, the logic behind working with all of them is mostly the same.

In our examples, we will look at working the Microsoft SQL Server as our database. For learning purposes, one can download and use the **Microsoft SQL Server Express Edition**, which is a free database software provided by Microsoft.

In working with databases, the following are the concepts which are common to all databases.

1. **Connection** – To work with the data in a database, the first obvious step is the connection. The connection to a database normally consists of the below-mentioned parameters.
 1. **Database name or Data Source** – The first important parameter is the database name to which the connection needs to be established. Each connection can only work with one database at a time.
 2. **Credentials** – The next important aspect is the username and password which needs to be used to establish a connection to the database. It ensures that the username and password have the necessary privileges to connect to the database.

3. **Optional parameters** – For each database type, you can specify optional parameters to provide more information on how .net should handle the connection to the database. For example, one can specify a parameter for how long the connection should stay active. If no operation is performed for a specific period of time, then the parameter would determine if the connection has to be closed.
2. **Selecting data from the database** – Once the connection has been established, the next important aspect is to fetch the data from the database. C# can execute 'SQL' select command against the database. The 'SQL' statement can be used to fetch data from a specific table in the database.
3. **Inserting data into the database** – C# can also be used to insert records into the database. Values can be specified in C# for each row that needs to be inserted into the database.
4. **Updating data into the database** – C# can also be used to update existing records into the database. New values can be specified in C# for each row that needs to be updated into the database.
5. **Deleting data from a database** – C# can also be used to delete records into the database. Select commands to specify which rows need to be deleted can be specified in C#.

Ok, now that we have seen the theory of each operation, let's jump into the further sections to look at how we can perform database operations in C#.

SQL Command in c#

SqlCommand in C# allow the user to query and send the commands to the database. SQL command is specified by the SQL connection object. Two methods are used, ExecuteReader method for results of query and ExecuteNonQuery for insert, Update, and delete commands. It is the method that is best for the different commands.

How to connect C# to Database

Let's now look at the code, which needs to be kept in place to create a connection to a database. In our example, we will connect to a database which has the name of Demodb. The credentials used to connect to the database are given below

- **Username – sa**
- **Password – demo123**

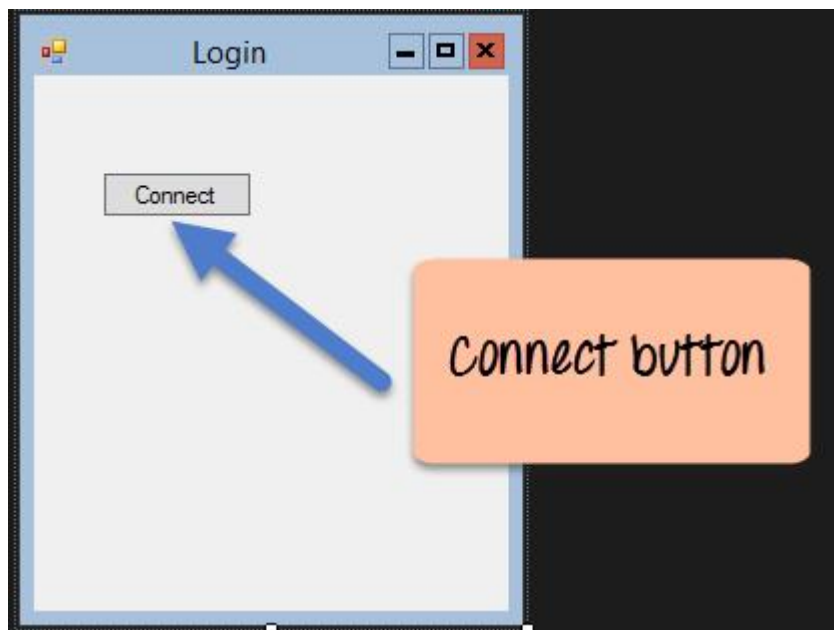
We will see a simple [Windows forms application](#) to work with databases. We will have a simple button called "Connect" which will be used to connect to the database.

So let's follow the below steps to achieve this

Step 1) The first step involves the creation of a new project in Visual Studio. After launching Visual Studio, you need to choose the menu option New->Project.

1. In the project dialog box, we can see various options for creating different types of projects in Visual Studio. Click the Windows option on the left-hand side.
2. When we click the Windows options in the previous step, we will be able to see an option for Windows Forms Application. Click this option.
3. We then give a name for the application which in our case is "DemoApplication". We also need to provide a location to store our application.
4. Finally, we click the 'OK' button to let Visual Studio to create our project.

Step 3) Now add a button from the toolbox to the Windows form. Put the text property of the Button as Connect. This is how it will look like



Step 4) Now double click the form so that an event handler is added to the code for the button click event. In the event handler, add the below code.

```
private void button1_Click(object sender, EventArgs e)
{
    string connetionString;
    SqlConnection cnn ;

    connetionString = @"Data Source=WIN-50GP30FG075;Initial Catalog=Demodb
;User ID=sa;Password=demo123";

    cnn = new SqlConnection(connetionString);
    cnn.Open();
    MessageBox.Show ("Connection Open ! ");
    cnn.Close();
}
```

Diagram illustrating the code for the button click event handler:

- 1. Variable declaration: `string connetionString; SqlConnection cnn ;`
- 2. Set connection string: `connetionString = @"Data Source=WIN-50GP30FG075;Initial Catalog=Demodb;User ID=sa;Password=demo123";`
- 3. Assign connection: `cnn = new SqlConnection(connetionString);`
- 4. Open connection: `cnn.Open();`
- 5. Close connection: `cnn.Close();`

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DemoApplication1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
            string connetionString;
            SqlConnection cnn;
            connetionString = @"Data Source=WIN-50GP30FGO75;Initial Catalog=Demodb;User
ID=sa;Password=demol23";
            cnn = new SqlConnection(connetionString);
            cnn.Open();
            MessageBox.Show("Connection Open  !");
            cnn.Close();
        }
    }
}

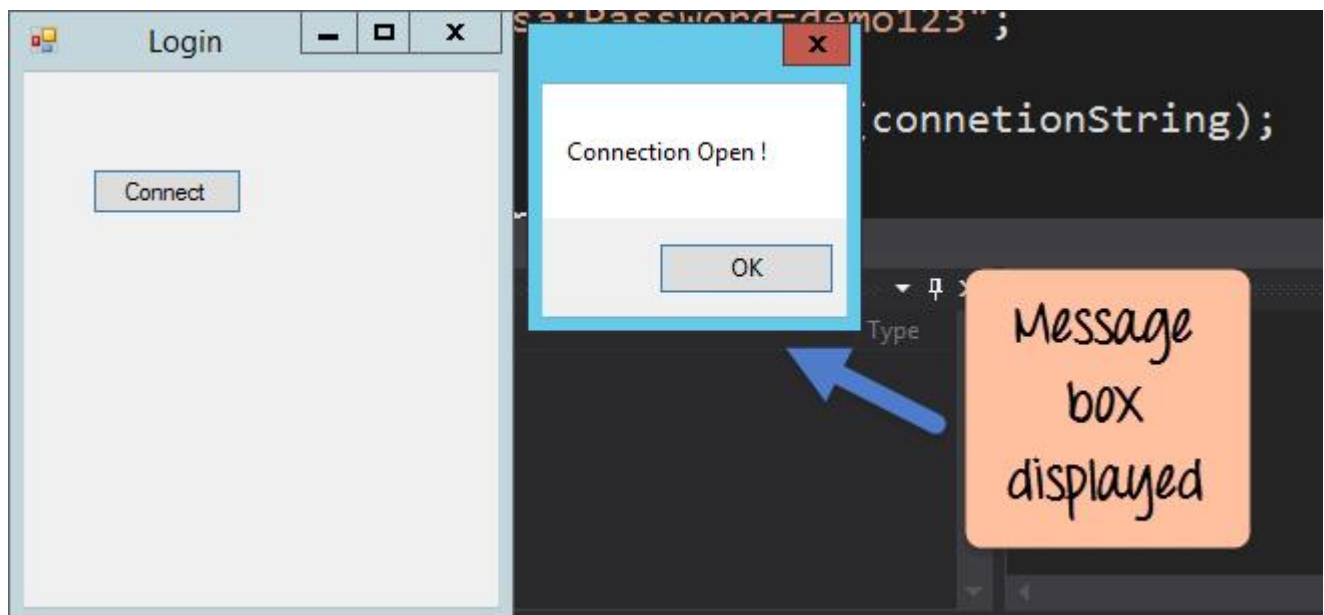
```

Code Explanation:-

1. The first step is to create variables, which will be used to create the connection string and the connection to the SQL Server database.
2. The next step is to create the connection string. The connecting string needs to be specified correctly for C# to understand the connection string. The connection string consists of the following parts
 1. Data Source – This is the name of the server on which the database resides. In our case, it resides on a machine called WIN- 50GP30FGO75.
 2. The Initial Catalog is used to specify the name of the database
 3. The UserID and Password are the credentials required to connect to the database.
3. Next, we assign the connecting string to the variable cnn. The variable cnn, which is of type SqlConnection is used to establish the connection to the database.
4. Next, we use the Open method of the cnn variable to open a connection to the database. We then just display a message to the user that the connection is established.
5. Once the operation is completed successfully, we then close the connection to the database. It is always a good practice to close the connection to the database if nothing else is required to be done on the database.

When the above code is set, and the project is executed using Visual Studio, you will get the below output. Once the form is displayed, click the Connect button.

Output:-



When you click on “connect” button, from the output, you can see that the database connection was established. Hence, the message box was displayed.

7.5 Reading and Writing elements to and from the database

In C#, you can read and write data to and from a database using various methods and libraries, with ADO.NET being a common choice for working with databases. Here, I'll provide an overview of how to read and write elements to and from a database using ADO.NET:

1. Reading Data from a Database:

To read data from a database in C# using ADO.NET, follow these steps:

i) Establish a Connection:

Create a connection to the database using a connection string. The connection string specifies the database type, location, authentication, and other connection parameters.

```
using System.Data.SqlClient; // For SQL Server
// ...
string connectionString = "Data Source=myServer;Initial
Catalog=myDatabase;User
ID=myUser;Password=myPassword;";
SqlConnection connection = new
SqlConnection(connectionString);
connection.Open(); // Open the database connection
```

ii) Create a SQL Command:

Define a SQL command to select data from the database. You can use a parameterized query to avoid SQL injection vulnerabilities.

```
SqlCommand command = new SqlCommand("SELECT *  
FROM MyTable WHERE SomeColumn = @ParamValue",  
connection);  
command.Parameters.AddWithValue("@ParamValue",  
someValue); // Replace with your parameter value
```

iii) Execute the Query:

Execute the SQL command using a data reader. The data reader allows you to read the results row by row.

```
SqlDataReader reader = command.ExecuteReader();  
while (reader.Read())  
{  
    // Access the data using reader methods (e.g.,  
reader["ColumnName"])  
    var value = reader["ColumnName"];  
    // Process the data  
}  
reader.Close(); // Close the reader when done
```

iv) Close the Connection:

Always close the database connection when you're finished with it.

```
connection.Close();
```


2. Writing Data to a Database:

To write data to a database in C# using ADO.NET, follow these steps:

i) Establish a Connection:

Create a connection to the database as described in the "Reading Data from a Database" section.

ii) Create a SQL Command:

Define a SQL command to insert, update, or delete data in the database. Again, you can use parameterized queries.

```
SqlCommand command = new SqlCommand("INSERT INTO  
MyTable (Column1, Column2) VALUES (@Value1,  
@Value2)", connection);  
command.Parameters.AddWithValue("@Value1", someValue1);  
command.Parameters.AddWithValue("@Value2", someValue2);
```

iii) Execute the Command:

Execute the SQL command to perform the database operation.

```
int rowsAffected = command.ExecuteNonQuery(); // Returns the  
number of affected rows
```

iv) Close the Connection:

Close the database connection when you're done.

```
connection.Close();
```

These are the fundamental steps for reading and writing data to and from a database using ADO.NET in C#. You can perform more complex operations, including updates and transactions, as needed for your application. Additionally, consider using Entity Framework or other ORMs for more high-level, object-oriented database access in C# to simplify database operations.